

FACULTY OF ENGINEERING AND TECHNOLOGY

Bachelor of Technology in Artificial Intelligence

Database Management System Laboratory
(03010504PC06)

SEMESTER III

Department of Artificial Intelligence and Data Science



Laboratory Manual



CERTIFICATE

*This is to certify that Ms Jaini Rasheshkumar Shahpatel with enrolment No. 2503031240107 has successfully completed her laboratory experiments in the **Database Management System Laboratory(03010504PC06)** from the department of Artificial Intelligence and Data Science during the academic year 2026-2027.*



Date of Submission: Staff In charge:

Head Of Department:

2503031240107

ORACLE Dev Gym

Certificate of Excellence

Jaini

for receiving a grade of 95% in
Databases for Developers: Foundations
taught by Chris Saxon



Presented by Oracle Corporation





CERTIFICATE OF COMPLETION

Jaini

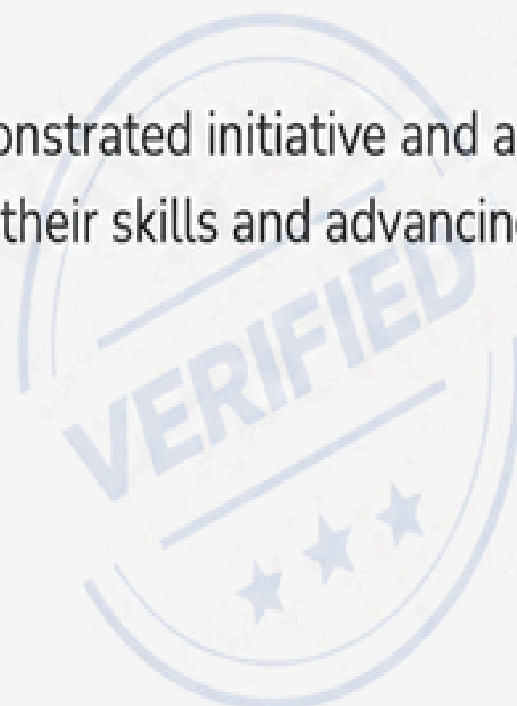
has successfully completed the online course:

Advance SQL

This professional has demonstrated initiative and a commitment to deepening their skills and advancing their career. Well done!

1st September 2026

Certificate code : 10563982



A handwritten signature in black ink.

Krishna Kumar
CEO, Simplilearn



kaggle

CERTIFICATE OF COMPLETION

Jaini

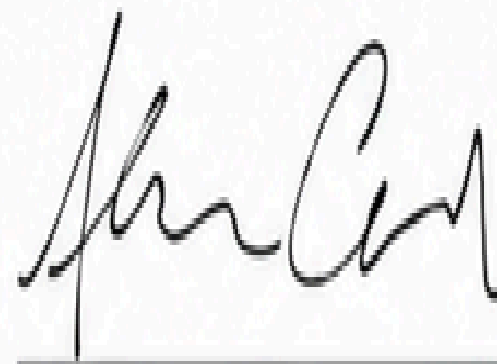
HAS SUCCESSFULLY COMPLETED THE COURSE

Intro to SQL

ON AUGUST 8, 2026



RACHAEL TATMAN, KAGGLE INSTRUCTOR



ALEXIS COOK, HEAD OF KAGGLE LEARN

TABLE OF CONTENTS

SR.NO	Experiment Title	Page No		Date of Performance	Date of Submission	Marks	Sign
		From	To				
1	To study DDL-create and DML-insert commands.	06	09	08-06-26	15-06-26		
2	Create the below given table and insert the data accordingly.	10	14	15-06-26	22-06-26		
3	To Perform various data manipulation commands, aggregate functions and sorting concept on all created tables.	15	19	22-06-26	13-07-26		
4.	To study Single-row functions	20	25	13-07-26	20-07-26		
5.	Displaying data from Multiple Tables (join)	26	30	20-07-26	27-07-26		
6.	To apply the concept of Aggregating Data using Group functions.	31	34	27-07-26	03-08-26		
7.	To solve queries using the concept of sub query.	35	38	03-08-26	10-08-26		
8.	Manipulating Data	39	42	10-08-26	31-08-26		
9.	TCL and DCL Commands in SQL	43	44	31-08-26	07-09-26		
10.	PL/SQL Block	45	46	07-09-26	14-09-26		

Enrollment No : 2503031240107

3

PRACTICAL: 1

Aim: To study DDL-create and DML-insert commands.

```
DROP TABLE IF EXISTS DEPOSIT;  
DROP TABLE IF EXISTS BRANCH;  
DROP TABLE IF EXISTS CUSTOMERS;  
DROP TABLE IF EXISTS BORROW;
```

```
CREATE TABLE DEPOSIT (  
  ACTNO VARCHAR2(5),  
  CNAME VARCHAR2(18),  
  BNAME VARCHAR2(18),  
  AMOUNT NUMBER(8,2),  
  ADATE DATE  
);
```

```
CREATE TABLE BRANCH (  
  BNAME VARCHAR2(18),  
  CITY VARCHAR2(18)  
);
```

```
CREATE TABLE CUSTOMERS (  
  CNAME VARCHAR2(19),  
  CITY VARCHAR2(18)  
);
```

```
CREATE TABLE BORROW (  
  LOANNO VARCHAR2(5), CNAME VARCHAR2(18),  
  BNAME VARCHAR2(18),  
  AMOUNT NUMBER(8,2)  
);
```

-- Deposit Entries

```
INSERT INTO DEPOSIT VALUES ('100','ANIL','VRCE',1000.00, '1995-03-01');
```

```
INSERT INTO DEPOSIT VALUES ('101','SUNIL','AJNI',5000.00, '1996-01-04');
```

```
INSERT INTO DEPOSIT VALUES ('102','MEHUL','KAROLBAGH',3500.00, '1995-11-17');
```

```
INSERT INTO DEPOSIT VALUES ('104','MADHURI','CHANDI',1200.00, '1995-12-17');
```

-- Branch Entries

```
INSERT INTO BRANCH VALUES ('VRCE','NAGPUR');
```

```
INSERT INTO BRANCH VALUES ('AJNI','NAGPUR');
```

```
INSERT INTO BRANCH VALUES ('KAROLBAGH','DELHI');
```

```
INSERT INTO BRANCH VALUES ('CHANDI','DELHI');
```

-- Customer Entries

```
INSERT INTO CUSTOMERS VALUES ('ANIL','CALCUTTA');
```

```
INSERT INTO CUSTOMERS VALUES ('SUNIL','DELHI');
```

```
INSERT INTO CUSTOMERS VALUES ('MEHUL','BARODA');
```

```
INSERT INTO CUSTOMERS VALUES ('MANDAR','PATNA');
```

-- Borrow Entries

```
INSERT INTO BORROW VALUES ('201','ANIL','VRCE',1000.00);
```

```
INSERT INTO BORROW VALUES ('206','MEHUL','AJNI',5000.00);
```

```
INSERT INTO BORROW VALUES ('311','SUNIL','DHARAMPETH',3000.00);
```

```
INSERT INTO BORROW VALUES ('321','MADHURI','ANDHERI',2000.00);
```

Query:

1. Describe deposit, branch.

Syntax: SELECT * FROM DEPOSIT;

ACTNO	CNAME	BNAME	AMOUNT	ADATE
100	ANIL	VRCE	1000	1995-03-01
101	SUNIL	AJNI	5000	1996-01-04
102	MEHUL	KAROLBAGH	3500	1995-11-17
104	MADHURI	CHANDI	1200	1995-12-17

Syntax: SELECT*FROM BRANCH;

BNAME	CITY
VRCE	NAGPUR
AJNI	NAGPUR
KAROLBAGH	DELHI
CHANDI	DELHI

2. Describe borrow, customers

Syntax: SELECT*FROM BORROW;

LOANNO	CNAME	BNAME	AMOUNT
201	ANIL	VRCE	1000
206	MEHUL	AJNI	5000
311	SUNIL	DHARAMPETH	3000
321	MADHURI	ANDHERI	2000

Syntax: SELECT*FROM CUSTOMERS;

CNAME	CITY
ANIL	CALCUTTA
SUNIL	DELHI
MEHUL	BARODA
MANDAR	PATNA

3. List all data from table DEPOSIT

Syntax: SELECT * FROM DEPOSIT;

ACTNO	CNAME	BNAME	AMOUNT	ADATE
100	ANIL	VRCE	1000	1995-03-01
101	SUNIL	AJNI	5000	1996-01-04
102	MEHUL	KAROLBAGH	3500	1995-11-17
104	MADHURI	CHANDI	1200	1995-12-17

4. List all data from table BORROW

Syntax: SELECT * FROM BORROW;

LOANNO	CNAME	BNAME	AMOUNT
201	ANIL	VRCE	1000
206	MEHUL	AJNI	5000
311	SUNIL	DHARAMPETH	3000
321	MADHURI	ANDHERI	2000

5. List all data from table CUSTOMERS

Syntax: SELECT * FROM CUSTOMERS;

CNAME	CITY
ANIL	CALCUTTA
SUNIL	DELHI
MEHUL	BARODA
MANDAR	PATNA

6. List all data from table BRANCH

Syntax: SELECT * FROM BRANCH;

BNAME	CITY
VRCE	NAGPUR
AJNI	NAGPUR
KAROLBAGH	DELHI
CHANDI	DELHI

7. Give account no and amount of depositors

Syntax: SELECT ACTNO, AMOUNT FROM DEPOSIT;

ACTNO	AMOUNT
100	1000
101	5000
102	3500
104	1200

8. Give name of depositors having amount greater than 4000

Syntax: SELECT CNAME FROM DEPOSIT WHERE AMOUNT > 4000;

Output
CNAME
SUNIL

9. Give name of customers who opened account after date '1-12-96'

Syntax: SELECT CNAME FROM DEPOSIT
WHERE ADATE > '1995-12-01';

CNAME
SUNIL
MADHURI

PRACTICAL: 2

Aim: Create the below given table and insert the data accordingly.

-- Drop tables if they already exist to prevent duplicate table errors

```
DROP TABLE IF EXISTS Employee;
```

```
DROP TABLE IF EXISTS Job;
```

```
DROP TABLE IF EXISTS Deposit;
```

-- Create Table Statements (Using SQLite compatible

```
datatypes) CREATE TABLE Job (  job_id VARCHAR(15),
```

```
job_title VARCHAR(30),  min_sal DECIMAL(7,2),
```

```
    max_sal DECIMAL(7,2)
```

```
);
```

```
CREATE TABLE Employee
```

```
(  emp_no INT,
```

```
emp_name VARCHAR(30),
```

```
    emp_sal DECIMAL(8,2),
```

```
    emp_comm
```

```
DECIMAL(6,1),
```

```
    dept_no INT
```

```
);
```

```
CREATE TABLE Deposit (
```

```
actno VARCHAR(5),
```

```
cname VARCHAR(20),
```

```
bname VARCHAR(20),
```

```
amount DECIMAL(8,2),
```

```
    bdate TEXT
```

```
);
```

-- Inserting values into the Job table

```
INSERT INTO Job (job_id, job_title, min_sal, max_sal) VALUES ('IT_PROG',
```

```
'Programmer', 4000, 10000);
```

```
INSERT INTO Job (job_id, job_title, min_sal, max_sal) VALUES ('MK_MGR', 'Marketing
manager', 9000, 15000);
```

```
INSERT INTO Job (job_id, job_title, min_sal, max_sal) VALUES ('FI_MGR', 'Finance
manager', 8200, 12000);
```

-- Inserting values into the Employee table

```
INSERT INTO Employee (emp_no, emp_name, emp_sal, emp_comm, dept_no) VALUES
(101, 'Smith', 800, 20, NULL);
```

```
INSERT INTO Employee (emp_no, emp_name, emp_sal, emp_comm, dept_no) VALUES
(102, 'Snehal', 1600, 300, 25);
```

```
INSERT INTO Employee (emp_no, emp_name, emp_sal, emp_comm, dept_no) VALUES
(103, 'Adama', 1100, 0, 20);
```

```
INSERT INTO Employee (emp_no, emp_name, emp_sal, emp_comm, dept_no) VALUES
(108, 'Sparks', 3200, 150, NULL);
```

-- Inserting values into the Deposit table (Fixed: Removed TO_DATE function)

```
INSERT INTO Deposit (actno, cname, bname, amount, bdate) VALUES ('101', 'Anil',
'VRCE', 1000.00, '1995-03-01');
```

```
INSERT INTO Deposit (actno, cname, bname, amount, bdate) VALUES ('102', 'Sunil',
'AJNI', 5000.00, '1996-01-04');
```

```
INSERT INTO Deposit (actno, cname, bname, amount, bdate) VALUES ('103', 'Mehula',
'DARBHANGA', 3500.00, '1995-07-15');
```

Query:

1. Retrieve all data from employee, jobs and deposit.

Syntax: SELECT * FROM Employee;

emp_no	emp_name	emp_sal	emp_comm	dept_no
101	Smith	800	20	
102	Snehal	1600	300	25
103	Adama	1100	0	20

Syntax: SELECT * FROM Job;

job_id	job_title	min_sal	max_sal
IT_PROG	Programmer	4000	10000
MK_MGR	Marketing manager	9000	15000
FI_MGR	Finance manager	8200	12000

Syntax: SELECT * FROM Deposit;

actno	cname	bname	amount	bdate
101	Anil	VRCE	1000	1995-03-01
102	Sunil	AJNI	5000	1996-01-04
103	Mehula	DARBHANGA	3500	1995-07-15

2. Give details of account no. and deposited rupees of customers having account opened between dates 01-01-06 and 25-07-06.

Syntax: SELECT actno, amount
FROM Deposit
WHERE bdate BETWEEN '1995-03-01' AND '1996-01-04';

actno	amount
101	1000
102	5000
103	3500

3.Display all jobs with minimum salary is greater than 4000

Syntax: SELECT *
FROM Job
WHERE min_sal > 4000;

job_id	job_title	min_sal	max_sal
MK_MGR	Marketing manager	9000	15000
FI_MGR	Finance manager	8200	12000

4. Display name and salary of employee whose department no is 20. Give alias name to name of employee.

Syntax: SELECT emp_name AS Employee_Name, emp_sal
FROM Employee
WHERE dept_no = 20;

Output	
Employee_Name	emp_sal
Adama	1100

5. Display employee no,name and department details of those employee whose department lies in(10,20)

Syntax: SELECT emp_no, emp_name, dept_no

FROM Employee

WHERE dept_no IN (10, 20);

Output		
emp_no	emp_name	dept_no
103	Adama	20

6. To study various options of LIKE predicate

Syntax: SELECT * FROM Employee WHERE emp_name LIKE 'A%';

Output				
emp_no	emp_name	emp_sal	emp_comm	dept_no
103	Adama	1100	0	20

Syntax: SELECT * FROM Employee WHERE emp_name LIKE '%a';

Output				
emp_no	emp_name	emp_sal	emp_comm	dept_no
103	Adama	1100	0	20

7. Display all employee whose name start with 'A' and third character is 'a'.

Syntax: SELECT * FROM Employee WHERE emp_name LIKE 'A_a%';

Output				
emp_no	emp_name	emp_sal	emp_comm	dept_no
103	Adama	1100	0	20

8. Display name, number and salary of those employees whose name is 5 characters long and first three characters are 'Ani'.

Syntax: SELECT emp_name, emp_no, emp_sal FROM Employee WHERE emp_name LIKE 'Ani_';

Output

SQL query successfully executed. However, the result set is empty.

9. Display the non-null values of employees and also employee name second character should be 'n' and string should be 5 character long.

Syntax: SELECT emp_name, emp_no, emp_sal
FROM Employee
WHERE emp_name IS NOT NULL
AND emp_name LIKE '_n%';

emp_name	emp_no	emp_sal
Snehal	102	1600

10. Display the null values of employee and also employee name's third character should be 'a'.

Syntax: SELECT emp_no, emp_name, emp_sal, emp_comm, dept_no
FROM Employee
WHERE (emp_comm IS NULL OR dept_no IS NULL)
AND emp_name LIKE '__a%';

emp_no	emp_name	emp_sal	emp_comm	dept_no
108	Sparks	3200	150	

11. What will be output if you are giving LIKE predicate as '%_%' ESCAPE '\'

Syntax: SELECT job_id FROM Job WHERE job_id LIKE '%_%' ESCAPE '\';

job_id
IT_PROG
MK_MGR
FI_MGR

Practical 3

Aim: To Perform various data manipulation commands, aggregate functions and sorting concept on all created tables.

-- Drop tables if they already exist to prevent duplicate table errors

DROP TABLE IF EXISTS Employee;

DROP TABLE IF EXISTS Job;

DROP TABLE IF EXISTS Deposit;

DROP TABLE IF EXISTS Loan;

-- Create Table Statements (Using SQLite compatible datatypes)
CREATE TABLE Job (
 job_id VARCHAR(15),
 job_title VARCHAR(30),
 min_sal DECIMAL(7,2),
 max_sal DECIMAL(7,2)
);

CREATE TABLE Employee (
 emp_no INT,
 emp_name VARCHAR(30),
 emp_sal DECIMAL(8,2),
 emp_comm DECIMAL(6,1),
 dept_no INT
);

CREATE TABLE Deposit (
 actno VARCHAR(5),
 cname VARCHAR(20),
 bname VARCHAR(20),
 amount DECIMAL(8,2),
 bdate TEXT
);

CREATE TABLE Loan (
 loanno VARCHAR(5),
 cname VARCHAR(20),

```
bname VARCHAR(20),  
amount DECIMAL(8,2) ); -
```

- Inserting values into the
Job table

```
INSERT INTO Job (job_id, job_title, min_sal, max_sal) VALUES ('IT_PROG', 'Programmer',  
4000, 10000);
```

```
INSERT INTO Job (job_id, job_title, min_sal, max_sal) VALUES ('MK_MGR', 'Marketing  
manager', 9000, 15000);
```

```
INSERT INTO Job (job_id, job_title, min_sal, max_sal) VALUES ('FI_MGR', 'Finance  
manager', 8200, 12000);
```

-- Inserting values into the Employee table

```
INSERT INTO Employee (emp_no, emp_name, emp_sal, emp_comm, dept_no) VALUES  
(101, 'Smith', 800, 20, NULL);
```

```
INSERT INTO Employee (emp_no, emp_name, emp_sal, emp_comm, dept_no) VALUES  
(102, 'Snehal', 1600, 300, 25);
```

```
INSERT INTO Employee (emp_no, emp_name, emp_sal, emp_comm, dept_no) VALUES  
(103, 'Adama', 1100, 0, 20);
```

```
INSERT INTO Employee (emp_no, emp_name, emp_sal, emp_comm, dept_no) VALUES  
(109, 'Anand', 2500, 100, 20);
```

```
INSERT INTO Employee (emp_no, emp_name, emp_sal, emp_comm, dept_no) VALUES  
(108, 'Sparks', 3200, 150, NULL);
```

-- Inserting values into the Deposit table (Fixed: Removed TO_DATE function)

```
INSERT INTO Deposit (actno, cname, bname, amount, bdate) VALUES ('101', 'Anil', 'VRCE',  
1000.00, '1995-03-01');
```

```
INSERT INTO Deposit (actno, cname, bname, amount, bdate) VALUES ('102', 'Sunil', 'AJNI',  
5000.00, '1996-01-04');
```

```
INSERT INTO Deposit (actno, cname, bname, amount, bdate) VALUES ('103', 'Mehula',  
'DARBHANGA', 3500.00, '1995-07-15');
```

```
INSERT INTO Loan (loanno, cname, bname, amount) VALUES ('L101', 'Anil', 'VRCE',  
10000.00);
```

```
INSERT INTO Loan (loanno, cname, bname, amount) VALUES ('L102', 'Sunil', 'Karolbagh',  
5000.00); INSERT INTO Loan (loanno, cname, bname, amount) VALUES ('L103', 'Jay',  
'VRCE',
```

```
12500.00); INSERT INTO Loan (loanno, cname, bname, amount) VALUES ('L104', 'Mandar',  
'AJNI', 3000.00); INSERT INTO Loan (loanno, cname, bname, amount) VALUES ('L105',  
'Pramod', 'Karolbagh', 9000.00);
```

Query:

1. List total deposit from deposit.

```
Syntax: SELECT SUM (amount) AS Total_Deposit  
FROM Deposit;
```

Total_Deposit
9500

2. List total loan from karolbagh branch

```
Syntax: SELECT SUM (amount) AS Total_Loan  
FROM Loan WHERE bname = 'Karolbagh';
```

Total_Loan
14000

3. Give maximum loan from branch vrce.

```
Syntax: SELECT MAX (amount) AS  
Maximum_Loan FROM Loan
```

```
WHERE bname = 'VRCE';
```

Maximum_Loan
12500

4. Count total number of customers

Syntax:

```
SELECT COUNT (DISTINCT cname) AS Total_Customers  
FROM Deposit;
```

Total_Customers
3

5. Count total number of customer' s cities.

Syntax: SELECT COUNT(DISTINCT bname) AS Total_Cities FROM Deposit;

Total_Cities
3

6. Create table supplier from employee with all the columns.

Syntax: CREATE TABLE supplier AS
SELECT * FROM Employee;

Supplier

emp_no	emp_name	emp_sal	emp_comm	dept_no
101	Smith	800	20	
102	Snehal	1600	300	25
103	Adama	1100	0	20
108	Sparks	3200	150	

7. Create table sup1 from employee with first two columns.

Syntax: CREATE TABLE sup1 AS
SELECT emp_no, emp_name
FROM Employee;

Sup1

emp_no	emp_name
101	Smith
102	Snehal
103	Adama
108	Sparks

8. Create table sup2 from employee with no data

Syntax: CREATE TABLE sup2 AS
SELECT * FROM Employee
WHERE 1 = 0;

Sup2

emp_no	emp_name	emp_sal	emp_comm	dept_no
empty				

9. Insert the data into sup2 from employee whose second character should be 'n' and string should be 5 characters long in employee name field.

Syntax: INSERT INTO sup2
SELECT * FROM Employee
WHERE emp_name LIKE '_n___';

Sup2

emp_no	emp_name	emp_sal	emp_comm	dept_no
109	Anand	2500	100	20

10. Delete all the rows from sup1.

Syntax: DELETE FROM sup1;

Sup1

emp_no	emp_name
empty	

11. Delete the detail of supplier whose sup_no is 103.

Syntax: DELETE FROM supplier

WHERE emp_no = 103;

Supplier

emp_no	emp_name	emp_sal	emp_comm	dept_no
101	Smith	800	20	
102	Snehal	1600	300	25
108	Sparks	3200	150	

12. Rename the table sup2.

Syntax: ALTER TABLE sup2 RENAME TO sup2_new;

Sup2_new

emp_no	emp_name	emp_sal	emp_comm	dept_no
109	Anand	2500	100	20

13. Update the value dept_no to 10 where second character of emp. name is ' m' .

Syntax: UPDATE Employee

SET dept_no = 10

WHERE emp_name LIKE '_m%';

Employee

emp_no	emp_name	emp_sal	emp_comm	dept_no
101	Smith	800	20	10
102	Snehal	1600	300	25
103	Adama	1100	0	20
109	Anand	2500	100	20
108	Sparks	3200	150	

Practical 4

Aim: To study Single-row functions

-- Drop tables if they already exist to prevent duplicate table errors

DROP TABLE IF EXISTS Travel;

DROP TABLE IF EXISTS Sailors;

DROP TABLE IF EXISTS Managers;

DROP TABLE IF EXISTS Products;

DROP TABLE IF EXISTS Boats;

DROP TABLE IF EXISTS Clients;

DROP TABLE IF EXISTS Employee;

-- Create Table Statements (SQLite compatible types)

CREATE TABLE Sailors (

SailorID INT PRIMARY KEY,

Name VARCHAR(30),

Age INT,

Rating INT,

Experience INT

);

CREATE TABLE Managers (

ManagerID INT PRIMARY KEY,

Name VARCHAR(30)

);

CREATE TABLE Products (

ProductID INT PRIMARY KEY,

ProductName VARCHAR(50),

Price DECIMAL(10,2)

);

CREATE TABLE Boats (

BoatID INT PRIMARY KEY,

```
BoatName VARCHAR(30),  
Color VARCHAR(20)  
);
```

```
CREATE TABLE Travel (  
    TravelID INT PRIMARY KEY,  
    SailorID INT,  
    TravelDate TEXT, -- Modified to TEXT for SQLite dates  
    FOREIGN KEY (SailorID) REFERENCES Sailors(SailorID)  
);
```

```
CREATE TABLE Clients (  
    Client_ID INT PRIMARY KEY,  
    Name VARCHAR(255),  
    Balance DECIMAL(10,2),  
    City VARCHAR(100),  
    Email VARCHAR(255)  
);
```

```
CREATE TABLE Employee  
(    emp_no INT,  
    emp_name  
    VARCHAR(30),  
    emp_sal DECIMAL(8,2),  
    emp_comm  
    DECIMAL(6,1),  
    dept_no INT  
);
```

```
-- Inserting values into Sailors  
INSERT INTO Sailors VALUES (1, 'John Johnson', 35, 4, 5);  
INSERT INTO Sailors VALUES (2, 'Sarah Lee', 28, 5, 3);  
INSERT INTO Sailors VALUES (3, 'Mike Brown', 45, 3, 10);  
INSERT INTO Sailors VALUES (4, 'Laura White', 32, 4, 6);  
INSERT INTO Sailors VALUES (5, 'David Clark', 27, 4, 4);
```

-- Inserting values into Managers

```
INSERT INTO Managers VALUES (1, 'Linda Green');  
INSERT INTO Managers VALUES (2, 'James Smith');  
INSERT INTO Managers VALUES (3, 'Susan Johnson');  
INSERT INTO Managers VALUES (4, 'Peter Brown');  
INSERT INTO Managers VALUES (5, 'Laura White');
```

-- Inserting values into Products

```
INSERT INTO Products VALUES (1, 'Boat Cleaner', 25);  
INSERT INTO Products VALUES (2, 'Life Jacket', 15);  
INSERT INTO Products VALUES (3, 'Navigation Map', 30);  
INSERT INTO Products VALUES (4, 'Boat Engine Oil', 50);  
INSERT INTO Products VALUES (5, 'Sail Repair Kit', 40);
```

-- Inserting values into Boats

```
INSERT INTO Boats VALUES (1, 'Ocean Breeze', 'Blue');  
INSERT INTO Boats VALUES (2, 'Sea Explorer', 'Green');  
INSERT INTO Boats VALUES (3, 'Wave Rider', 'Red');  
INSERT INTO Boats VALUES (4, 'SailMaster', 'White');  
INSERT INTO Boats VALUES (5, 'Deep Sea Hunter', 'Black');
```

-- Inserting values into Travel (Clean string dates)

```
INSERT INTO Travel VALUES (1, 1, '2023-05-01');  
INSERT INTO Travel VALUES (2, 2, '2023-05-02');  
INSERT INTO Travel VALUES (3, 3, '2023-05-03');  
INSERT INTO Travel VALUES (4, 4, '2023-05-04');  
INSERT INTO Travel VALUES (5, 5, '2023-05-05');
```

-- Inserting values into Clients

```
INSERT INTO Clients VALUES (1, 'SOPHIA', 5000, 'MUMBAI', 'sophia@email.com');  
INSERT INTO Clients VALUES (2, 'ARJUN', 3000, 'DELHI', 'arjun@email.com');  
INSERT INTO Clients VALUES (3, 'PRIYA', 7000, 'KOLKATA', 'priya@email.com');  
INSERT INTO Clients VALUES (4, 'DEVAN', 4500, 'MUMBAI', 'devansh@email.com');  
INSERT INTO Clients VALUES (5, 'MIRA', 8000, 'BANGALORE', 'mira@email.com');
```

-- Inserting values into Employee

```
INSERT INTO Employee VALUES (101, 'Smith', 800, 20, 20);
INSERT INTO Employee VALUES (102, 'Snehal', 1600, 300, 25);
INSERT INTO Employee VALUES (103, 'Adama', 1100, 0, 20);
INSERT INTO Employee VALUES (104, 'Aman', 3000, 15, 10);
INSERT INTO Employee VALUES (105, 'Anita', 5000, 50000, 10);
INSERT INTO Employee VALUES (106, 'Sneha', 2450, 24500, 10);
INSERT INTO Employee VALUES (107, 'Anamika', 2975, 20000, 30);
```

Query:

1. Write a query to display the current date. Label the column Date

Syntax: `SELECT DATE('now') AS Date;`

Output

Date
2026-08-09

2. For each employee, display the employee number, job, salary, and salary increased by 15% and expressed as a whole number. Label the column New Salary

Syntax: `SELECT emp_no,
 emp_sal,
 ROUND(emp_sal * 1.15, 0) AS "New Salary"
 FROM Employee;`

emp_no	emp_sal	New Salary
101	800	920
102	1600	1840
103	1100	1265
104	3000	3450
105	5000	5750
106	2450	2818
107	2975	3421

3. Modify your query no 4.(2) to add a column that subtracts the old salary from the new salary. Label the column Increase

Syntax: SELECT emp_no,

emp_no	emp_sal	New Salary	Increase
101	800	920	120
102	1600	1840	240
103	1100	1265	165
104	3000	3450	450
105	5000	5750	750
106	2450	2818	368
107	2975	3421	446

emp_sal,

ROUND(emp_sal * 1.15, 0) AS "New Salary",

(ROUND(emp_sal * 1.15, 0) - emp_sal) AS Increase

FROM Employee;

4. Write a query that displays the employee's names with the first letter capitalized and all other letters lowercase, and the length of the names, for all employees whose name starts with J, A, or M. Give each column an appropriate label. Sort the results by the employees' last names

Syntax: SELECT UPPER(SUBSTR(emp_name, 1, 1)) || LOWER(SUBSTR(emp_name, 2)) AS "Employee Name",

LENGTH(emp_name) AS "Name Length" FROM Employee WHERE emp_name LIKE 'J%' OR emp_name LIKE 'A%'

Output

Employee Name	Name Length
Adam	4
Aman	4
Anita	5

5. Write a query that produces the following for each employee: <emp_name> earns <emp_sal> monthly.

Syntax: SELECT emp_name || ' earns ' || emp_sal || ' monthly' AS "Monthly Earnings"
FROM Employee;

Output

Monthly Earnings
Smith earns 800 monthly
Snehal earns 1600 monthly
Adam earns 1100 monthly
Aman earns 3000 monthly
Anita earns 5000 monthly

“Like” Queries:

1. Display all customers whose name start with ‘ M’ .

Syntax: SELECT * FROM Clients WHERE Name LIKE 'M%';

Client_ID	Name	Balance	City	Email
5	MIRA	8000	BANGALORE	mira@email.com

2.Display the details of sailors whose name is minimum 6 characters long.

Syntax: SELECT * FROM Sailors WHERE LENGTH(Name) >= 6;

SailorID	Name	Age	Rating	Experience
1	John Johnson	35	4	5
2	Sarah Lee	28	5	3
3	Mike Brown	45	3	10
4	Laura White	32	4	6
5	David Clark	27	4	4

3.List all customers whose city contains ‘ a’ as second character.

Syntax: SELECT * FROM Clients WHERE City LIKE '_A%';

Client_ID	Name	Balance	City	Email
5	MIRA	8000	BANGALORE	mira@email.com

4.Display the details of sailors whose name is minimum 6 characters long.

Syntax: SELECT * FROM Sailors WHERE NAME LIKE '_____ %';

Output

SailorID	Name	Age	Rating	Experience
1	John Johnson	35	4	5
2	Sarah Lee	28	5	3
3	Mike Brown	45	3	10
4	Laura White	32	4	6

5.Display the details of Employees whose name starts with ‘ S’ .

Syntax: SELECT * FROM Employee WHERE emp_name LIKE 'S%';

Output

emp_no	emp_name	emp_sal	emp_comm	dept_no
101	Smith	800		10
102	Snehal	1600	300	25

6. List the details of the boat ending with ‘ e’ .

Syntax: SELECT * FROM Boats WHERE BoatName LIKE '%e';

Output

BoatID	BoatName	Color
1	Ocean Breeze	Blue

7. List the details of clients having ‘ i’ as a 3rd character in his/her name.

SELECT * FROM Clients WHERE client_name LIKE '__I%';

Output

Client_ID	Client_Name	Balance	Due_Balance	City	State	Pincode	Email
3	PRIYA	7000	2500	KOLKATA	WEST BENGAL	700001	priya@email.com

8. List Client Name, due balance and city whose pin code starts with 4.

SELECT client_name, due_balance, city FROM Clients WHERE pincode LIKE '4%';

Output

Client_Name	Due_Balance	City
SOPHIA	1200	MUMBAI
DEVAN	0	MUMBAI

9. List all customers whose city contains ‘ a’ as second character.

SELECT * FROM CUSTOMERS WHERE CITY LIKE '_a%';

Output

CNAME	CITY
ANIL	CALCUTTA
MEHUL	BARODA
MANDAR	PATNA

10. List client names and city whose state has ‘ a’ as fourth or fifth character.

SELECT client_name, city FROM Clients WHERE state LIKE '___a%' OR state LIKE '____a%';

Output

Client_Name	City
SOPHIA	MUMBAI
DEVAN	MUMBAI

Practical 5

Aim: Displaying data from Multiple Tables (join)

-- Drop tables in reverse order of dependencies to avoid constraint violations

DROP TABLE IF EXISTS Employees;

DROP TABLE IF EXISTS Departments;

DROP TABLE IF EXISTS Locations;

DROP TABLE IF EXISTS Borrower;

DROP TABLE IF EXISTS Depositor;

DROP TABLE IF EXISTS Account;

DROP TABLE IF EXISTS Customer;

DROP TABLE IF EXISTS Branch;

```
CREATE TABLE Branch (  
    branch_name VARCHAR(50) PRIMARY KEY,  
    branch_city VARCHAR(50)  
);
```

```
CREATE TABLE Customer (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(50),  
    living_city VARCHAR(50),  
    branch_name VARCHAR(50),  
    FOREIGN KEY (branch_name) REFERENCES Branch(branch_name)  
);
```

```
CREATE TABLE Locations (  
    location_id INT PRIMARY KEY,  
    city VARCHAR(50)  
);
```

```
CREATE TABLE Account (  
    account_no VARCHAR(20) PRIMARY KEY,
```

```
customer_id INT,  
balance DECIMAL(10,2),  
FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)  
);  
  
CREATE TABLE Depositor (  
customer_id INT, account_no VARCHAR(20),  
PRIMARY KEY (customer_id, account_no),  
FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),  
FOREIGN KEY (account_no) REFERENCES Account(account_no)  
);  
  
CREATE TABLE  
Borrower (  
customer_id INT,  
loan_no VARCHAR(20),  
PRIMARY KEY (customer_id, loan_no),  
FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)  
);  
  
CREATE TABLE Departments (  
department_id INT PRIMARY KEY,  
department_name VARCHAR(50),  
location_id INT,  
FOREIGN KEY (location_id) REFERENCES Locations(location_id)  
);  
  
CREATE TABLE Employees (  
employee_id INT PRIMARY  
KEY, first_name  
VARCHAR(50), last_name  
VARCHAR(50), job_id  
VARCHAR(30), manager_id  
INT,  
hire_date TEXT, -- Changed to TEXT for reliable SQLite date handling  
department_id INT,  
FOREIGN KEY (department_id) REFERENCES Departments(department_id),  
FOREIGN KEY (manager_id) REFERENCES Employees(employee_id)
```

);

-- 3. Insert Master Data

INSERT INTO Branch VALUES ('Main Branch', 'Nagpur');

INSERT INTO Branch VALUES ('West End', 'Mumbai');

INSERT INTO Customer VALUES (1, 'ANIL', 'Nagpur', 'Main Branch');

INSERT INTO Customer VALUES (2, 'SUNIL', 'Nagpur', 'Main Branch');

INSERT INTO Customer VALUES (3, 'AMIT', 'Mumbai', 'West End');

INSERT INTO Account VALUES ('ACC101', 1, 50000.00);

INSERT INTO Account VALUES ('ACC102', 2, 75000.00);

INSERT INTO Depositor VALUES (1, 'ACC101');

INSERT INTO Depositor VALUES (2, 'ACC102');

INSERT INTO Borrower VALUES (1, 'LOAN901');

INSERT INTO Locations VALUES (10, 'NEW YORK');

INSERT INTO Locations VALUES (20, 'CHICAGO');

INSERT INTO Departments VALUES (10, 'Accounting', 10);

INSERT INTO Departments VALUES (20, 'Research', 20);

INSERT INTO Departments VALUES (30, 'Sales', 20);

INSERT INTO Employees VALUES (7839, 'KING', 'KING', 'PRESIDENT', NULL, '1990-11-17', 10);

INSERT INTO Employees VALUES (7566, 'JONES', 'JONES', 'MANAGER', 7839, '1991-04-02', 20);

INSERT INTO Employees VALUES (7788, 'SCOTT', 'SCOTT', 'ANALYST', 7566, '1992-12-09', 20);

INSERT INTO Employees VALUES (7902, 'FORD', 'FORD', 'ANALYST', 7566, '1993-12-03', 20);

INSERT INTO Employees VALUES (7499, 'ALLEN', 'ALLEN', 'SALESMAN', 7566, '1991-02-20', 30);

Query:

1. Give details of customers ANIL.

Syntax: SELECT * FROM DEPOSIT WHERE CNAME LIKE '%ANIL%';

actno	cname	bname	amount	bdate
101	Anil	VRCE	1000	1995-03-01

2.Give name of customer who are borrowers and depositors and having living city nagpur

Syntax: SELECT customer_name

FROM Customer

WHERE living_city = 'Nagpur'

AND customer_id IN (SELECT customer_id FROM Depositor)

AND customer_id IN (SELECT customer_id FROM Borrower);

Output

customer_name
ANIL

3.Give city as their city name of customers having same living branch.

Syntax: SELECT DISTINCT Customer.living_city AS "City Name"

FROM Customer

JOIN Branch ON Customer.branch_name = Branch.branch_name

WHERE Customer.living_city = Branch.branch_city;

City Name
Nagpur
Mumbai

4. Write a query to display the last name, department number, and department name for all employees.

Syntax: SELECT E.last_name, E.department_id, D.department_name

FROM Employees E

JOIN Departments D ON E.department_id = D.department_id;

last_name	department_id	department_name
KING	10	Accounting
JONES	20	Research
SCOTT	20	Research
FORD	20	Research
ALLEN	30	Sales

5.Create a unique listing of all jobs that are in department 30. Include the location of the department in the output

Syntax: SELECT DISTINCT E.job_id, D.location_id

FROM Employees E

JOIN Departments D ON E.department_id = D.department_id

WHERE E.department_id = 30;

job_id	location_id
SALESMAN	20

6.Write a query to display the employee name, department number, and department name for all employees who work in NEW YORK.

Syntax: SELECT E.first_name, E.last_name, E.department_id, D.department_name

FROM Employees E

JOIN Departments D ON E.department_id = D.department_id

JOIN Locations L ON D.location_id = L.location_id

WHERE L.city = 'NEW YORK';

Output

first_name	last_name	department_id	department_name
KING	KING	10	Accounting

7.Display the employee last name and employee number along with their manager's last name and manager number.Label the columns Employee, Emp#, Manager, and Mgr#, respectively.

Create a query to display the name and hire date of any employee hired after employee SCOTT.

Syntax: SELECT e.last_name AS "Employee",

e.employee_id AS "Emp#",

```
m.last_name AS "Manager",  
m.employee_id AS "Mgr#"  
FROM Employees e  
JOIN Employees m ON e.manager_id = m.employee_id;
```

Employee	Emp#	Manager	Mgr#
JONES	7566	KING	7839
SCOTT	7788	JONES	7566
FORD	7902	JONES	7566
ALLEN	7499	JONES	7566

Practical 6

AIM: To apply the concept of Aggregating Data using Group functions.

```
DROP TABLE IF EXISTS employees;
```

```
DROP TABLE IF EXISTS deposit;
```

```
CREATE TABLE employees (
```

```
    emp_id INT PRIMARY KEY,
```

```
    emp_name VARCHAR(50),
```

```
    job VARCHAR(50),
```

```
    salary DECIMAL(10,2),
```

```
    hire_date DATE,
```

```
    dept_id INT
```

```
);
```

```
INSERT INTO employees (emp_id, emp_name, job, salary, hire_date, dept_id) VALUES
```

```
(101, 'Amit', 'PRESIDENT', 5000.00, '1995-01-10', 10),
```

```
(102, 'Sanjay', 'MANAGER', 3500.00, '1995-06-15', 10),
```

```
(103, 'Neha', 'ANALYST', 2800.00, '1996-03-22', 20),
```

```
(104, 'Rohan', 'ANALYST', 2900.00, '1997-11-05', 20),
```

```
(105, 'Kiran', 'CLERK', 1200.00, '1998-12-17', 10),
```

```
(106, 'Pooja', 'CLERK', 1500.00, '1996-05-19', 30),
```

```
(107, 'Vijay', 'SALESMAN', 1800.00, '1997-09-01', 30);
```

```
CREATE TABLE deposit (
```

```
    bname VARCHAR(50),
```

```
    cname VARCHAR(50),
```

```
    amount DECIMAL(10,2),
```

```
    a_date DATE,
```

```
    city VARCHAR(50)
```

```
);
```

```
INSERT INTO deposit (bname, cname, amount, a_date, city) VALUES
```

```
('Dharampeth', 'Anil', 6000.00, '1995-03-12', 'Nagpur'),
```

```
('Main', 'Sunil', 4500.00, '1996-02-15', 'Nagpur'),
```

```
('Chadani', 'Rahul', 8000.00, '1997-06-20', 'Bombay'),
```

```
('Borivali', 'Vijay', 3500.00, '1995-11-01', 'Bombay'),
```

```
('M G Road', 'Pramod', 2000.00, '1996-08-11', 'Nagpur'),
```

('Chadani', 'Ajay', 2500.00, '1998-01-05', 'Bombay');

Query:

1. List total deposit of customer having account date after 1-jan-96.

Syntax: SELECT SUM (amount) AS total_deposit
 FROM deposit
 WHERE a_date > '1996-01-01';

Output	
total_deposit	
17000	

2.List total deposit of customers living in city Nagpur.

Syntax: SELECT SUM (amount) AS total_deposit
 FROM deposit
 WHERE city = 'Nagpur';

Output	
total_deposit	
12500	

3. List maximum deposit of customers living in bombay.

Syntax: SELECT MAX (amount) AS max_deposit
 FROM deposit
 WHERE city = 'Bombay';

Output	
max_deposit	
8000	

4. Display the highest, lowest, sum, and average salary of all employees. Label the columns Maximum, Minimum, Sum, and Average, respectively. Round your results to the nearest whole number.

Syntax: SELECT
 ROUND(MAX(salary)) AS Maximum,
 ROUND(MIN(salary)) AS Minimum,
 ROUND(SUM(salary)) AS Sum,
 ROUND(AVG (salary)) AS Average
 FROM employees;

Output			
Maximum	Minimum	Sum	Average
5000	1200	18700	2671

5. Write a query that displays the difference between the highest and lowest salaries. Label the

column DIFFERENCE.

Syntax: SELECT (MAX(salary) - MIN(salary)) AS DIFFERENCE
FROM employees;

Output

DIFFERENCE
3800

6. Create a query that will display the total number of employees and, of that total, the number of employees hired in 1995, 1996, 1997, and 1998

Syntax: SELECT
COUNT(*) AS total_employees,
COUNT(CASE WHEN hire_date LIKE '1995%' THEN 1 END) AS "1995",
COUNT(CASE WHEN hire_date LIKE '1996%' THEN 1 END) AS "1996",
COUNT(CASE WHEN hire_date LIKE '1997%' THEN 1 END) AS "1997",
COUNT(CASE WHEN hire_date LIKE '1998%' THEN 1 END) AS "1998"
FROM employees;

Output

total_employees	1995	1996	1997	1998
7	2	2	2	1

7. Find the average salaries for each department without displaying the respective department numbers.

Syntax: SELECT AVG(salary) AS average_salary
FROM employees
GROUP BY dept_id;

Output

average_salary
3233.3333333333335
2850
1650

8. Write a query to display the total salary being paid to each job title, within each department.

Syntax: SELECT dept_id, job, SUM(salary) AS total_salary
FROM employees
GROUP BY dept_id, job;

Output		
dept_id	job	total_salary
10	CLERK	1200
10	MANAGER	3500
10	PRESIDENT	5000
20	ANALYST	5700
30	CLERK	1500

9. Find the average salaries > 2000 for each department without displaying the respective department numbers.

Syntax: SELECT AVG (salary) AS average_salary
FROM employees
GROUP BY dept_id
HAVING AVG (salary) > 2000;

Output	
average_salary	
3233.3333333333335	
2850	

10. Display the job and total salary for each job with a total salary amount exceeding 3000, in which excludes president and sorts the list by the total salary.

Syntax: SELECT job, SUM (salary) AS total_salary FROM employees WHERE job <> 'PRESIDENT'
GROUP BY job HAVING SUM (salary) > 3000 ORDER BY total_salary;

Output	
job	total_salary
MANAGER	3500
ANALYST	5700

11.List the branches having sum of deposit more than 5000 and located in city "bombay".

Syntax: SELECT bname, SUM (amount) AS total_depositFROM deposit
WHERE city = 'Bombay'
GROUP BY bname
HAVING SUM (amount) > 5000;

Output	
bname	total_deposit
Chadani	10500

Practical 7

AIM: To solve queries using the concept of sub query.

```
DROP TABLE IF EXISTS borrow;  
DROP TABLE IF EXISTS deposit;  
DROP TABLE IF EXISTS branch;  
DROP TABLE IF EXISTS customer;  
DROP TABLE IF EXISTS employees;  
DROP TABLE IF EXISTS departments;
```

```
CREATE TABLE departments (department_id INT PRIMARY KEY, department_name  
VARCHAR(50) NOT NULL);
```

```
INSERT INTO departments VALUES (10, 'Accounting'), (20, 'Research'), (30, 'Sales');
```

```
CREATE TABLE employees (employee_id INT PRIMARY KEY, last_name VARCHAR(50)  
NOT NULL, job_id VARCHAR(50), manager_id INT, hire_date DATE, salary DECIMAL(10, 2),  
department_id INT);
```

```
INSERT INTO employees VALUES  
(101, 'FORD', 'ANALYST', NULL, '2021-12-03', 3000.00, 20),  
(102, 'SCOTT', 'ANALYST', 101, '2022-12-09', 3000.00, 20),  
(103, 'SMITH', 'CLERK', 101, '2023-01-15', 800.00, 20),  
(104, 'MILLER', 'CLERK', 102, '2022-01-23', 1300.00, 10),  
(105, 'ADAMS', 'CLERK', 102, '2023-05-23', 1100.00, 20);
```

```
CREATE TABLE customer (cname VARCHAR(50) PRIMARY KEY, city VARCHAR(50) NOT  
NULL);
```

```
INSERT INTO customer VALUES ('SUNIL', 'Mumbai'), ('PRAMOD', 'Nagpur'), ('ANIL', 'Delhi'),  
('AMIT', 'Mumbai'), ('RAHUL', 'Nagpur');
```

```
CREATE TABLE branch (bname VARCHAR(50) PRIMARY KEY, bcity VARCHAR(50) NOT  
NULL);
```

```
INSERT INTO branch VALUES ('VRCE', 'Nagpur'), ('AJNI', 'Nagpur'), ('CHANDNI', 'Delhi'), ('DHARAMPETH', 'Nagpur');
```

```
CREATE TABLE deposit (actno VARCHAR(20) PRIMARY KEY, cname VARCHAR(50), bname VARCHAR(50), amount DECIMAL(10, 2));
```

```
INSERT INTO deposit VALUES ('D101','SUNIL','VRCE',5000.00), ('D102','ANIL','CHANDNI',3500.00), ('D103','PRAMOD','VRCE',1500.00), ('D104','AMIT','CHANDNI',2500.00), ('D105','RAHUL','AJNI',4000.00);
```

```
CREATE TABLE borrow (loanno VARCHAR(20) PRIMARY KEY, cname VARCHAR(50), bname VARCHAR(50), amount DECIMAL(10, 2));
```

```
INSERT INTO borrow VALUES ('L201','PRAMOD','VRCE',10000.00), ('L202','SUNIL','AJNI',5000.00), ('L203','RAHUL','DHARAMPETH',8000.00);
```

Query:

1. Write a query to display the last name and hire date of any employee in the same department as

SCOTT. Exclude SCOTT

Syntax: SELECT last_name, hire_date FROM employees

WHERE department_id = (SELECT department_id FROM employees WHERE last_name = 'SCOTT')

AND last_name <> 'SCOTT';

Output

last_name	hire_date
FORD	2021-12-03
SMITH	2023-01-15
ADAMS	2023-05-23

2. Give name of customers who are depositors having same branch city of mr. sunil.

Syntax: SELECT DISTINCT cname FROM deposit WHERE bname IN(SELECT bname FROM branch WHERE bcity=(SELECT bcity FROM branch WHERE bname=(SELECT bname FROM deposit WHERE cname='SUNIL')));

Output
cname
SUNIL
PRAMOD
RAHUL

3.Give deposit details and loan details of customer in same city where pramod is living.

Syntax: SELECT * FROM deposit WHERE cname IN(SELECT cname FROM customer WHERE city=(SELECT city FROM customer WHERE cname='PRAMOD'));

SELECT * FROM borrow WHERE cname IN(SELECT cname FROM customer WHERE city=(SELECT city FROM customer WHERE cname='PRAMOD'));

Output

actno	cname	bname	amount
D103	PRAMOD	VRCE	1500
D105	RAHUL	AJNI	4000

loanno	cname	bname	amount
L201	PRAMOD	VRCE	10000
L203	RAHUL	DHARAMPETH	8000

4. Create a query to display the employee numbers and last names of all employees who earn more than the average salary. Sort the results in ascending order of salary.

Syntax: SELECT employee_id, last_name, salary

FROM employees

WHERE salary > (SELECT AVG (salary) FROM employees)

ORDER BY salary ASC;

Output

employee_id	last_name	salary
101	FORD	3000
102	SCOTT	3000

5. Give names of depositors having same living city as mr. anil and having deposit amount greater than 2000

Syntax: SELECT DISTINCT cname

FROM deposit

WHERE amount > 2000

AND cname IN (

SELECT cname

```

FROM customer
WHERE city = (SELECT city FROM customer WHERE cname = 'ANIL')
);

```

Output

cname
ANIL

6.Display the last name and salary of every employee who reports to ford.

```

Syntax: SELECT last_name, salary FROM employees
WHERE manager_id = (SELECT employee_id FROM employees WHERE last_name =
'FORD');

```

Output

last_name	salary
SCOTT	3000
SMITH	800

7.Display the department number, name, and job for every employee in the Accounting department.

```

Syntax: SELECT e.department_id, d.department_name, e.job_id AS job
FROM employees e
JOIN departments d ON e.department_id = d.department_id
WHERE d.department_name = 'Accounting';

```

Output

department_id	department_name	job
10	Accounting	CLERK

8. List the name of branch having highest number of depositors.

```

Syntax: SELECT bname FROM deposit GROUP BY bname ORDER BY COUNT(cname) DESC
LIMIT 1;

```

Output

bname
VRCE

9.Give the name of cities where in which the maximum numbers of branches are located.

Syntax: SELECT bcity FROM branch GROUP BY bcity ORDER BY COUNT(*) DESC LIMIT 1;

Output

bcity
Nagpur

10. Give name of customers living in same city where maximum depositors are located.

Syntax: SELECT cname FROM customer WHERE city = (
SELECT city FROM customer WHERE cname = (
SELECT cname FROM deposit GROUP BY cname ORDER BY COUNT(*) DESC LIMIT 1
)
)

Output

cname
SUNIL
AMIT

Practical-8

AIM: Manipulating Data

-- 1. DROP EXISTING CONFLICTS

DROP TABLE IF EXISTS borrow;

DROP TABLE IF EXISTS deposit;

DROP TABLE IF EXISTS customer;

DROP TABLE IF EXISTS branch;

DROP TABLE IF EXISTS emp;

-- 2. SCHEMAS GENERATION

CREATE TABLE branch (

 bname VARCHAR(50) PRIMARY KEY,

 bcity VARCHAR(50) NOT NULL

);

CREATE TABLE customer (

 cname VARCHAR(50) PRIMARY KEY,

 city VARCHAR(50) NOT NULL

);

CREATE TABLE deposit (

 cname VARCHAR(50),

 bname VARCHAR(50),

 amount DECIMAL(10, 2) NOT NULL,

 PRIMARY KEY (cname, bname),

 FOREIGN KEY (cname) REFERENCES customer(cname) ON DELETE CASCADE,

 FOREIGN KEY (bname) REFERENCES branch(bname) ON DELETE CASCADE

);

CREATE TABLE borrow (

 loan_no INT PRIMARY KEY,

 cname VARCHAR(50),

 bname VARCHAR(50),

```

amount DECIMAL(10, 2) NOT NULL,
FOREIGN KEY (cname) REFERENCES customer(cname) ON DELETE CASCADE,
FOREIGN KEY (bname) REFERENCES branch(bname) ON DELETE CASCADE
);

CREATE TABLE emp (
    empno INT PRIMARY KEY,
    ename VARCHAR(50) NOT NULL,
    job VARCHAR(50) NOT NULL,
    deptno INT NOT NULL
);

-- 3. INITIAL MOCK RECORDS (Perfect for verifying your lab updates/deletions)
INSERT INTO branch (bname, bcity) VALUES ('vrce', 'nagpur'), ('main_street', 'bombay'),
('chandni_chowk', 'delhi');
INSERT INTO customer (cname, city) VALUES ('anil', 'nagpur'), ('sunil', 'nagpur'), ('vijay',
'nagpur'), ('rahul', 'bombay'), ('amit', 'delhi');
INSERT INTO deposit (cname, bname, amount) VALUES ('anil', 'vrce', 5000.00), ('sunil',
'vrce', 4000.00), ('vijay', 'main_street', 9000.00), ('rahul', 'main_street', 1500.00);
INSERT INTO borrow (loan_no, cname, bname, amount) VALUES (101, 'anil', 'vrce',
2000.00), (102, 'amit', 'chandni_chowk', 800.00), (103, 'rahul', 'chandni_chowk', 600.00);
INSERT INTO emp (empno, ename, job, deptno) VALUES (7788, 'scott', 'analyst', 20), (7844,
'turner', 'salesman', 30);

```

Query:

1. Give 10% interest to all depositors.

```

Syntax: UPDATE deposit
SET amount = amount * 1.10;
-- Just run a SELECT directly to view the changes
SELECT * FROM deposit;

```

Output		
cname	bname	amount
anil	vrce	5500
sunil	vrce	4400
vijay	main_street	9900
rahul	main_street	1650.00000000000002

2. Give 10% Interest to Depositors Having Branch 'VRCE'

Syntax: UPDATE deposit
SET amount = amount * 1.10
WHERE bname = 'vrce';

Output

SQL query successfully executed. However, the result set is empty.

3. Give 10% Interest to Depositors Living in Nagpur and Having Branch City Bombay

Syntax: UPDATE deposit
SET amount = amount * 1.10
WHERE cname IN (SELECT cname FROM customer WHERE city = 'nagpur')
AND bname IN (SELECT bname FROM branch WHERE bcity = 'bombay');

Output

SQL query successfully executed. However, the result set is empty.

4. Change Department Number of Employees Having the Same Job as Employee 7788 to Employee 7844's Department

Syntax: UPDATE emp
SET deptno = (SELECT deptno FROM emp WHERE empno = 7844)
WHERE empno = 7788;

Output

SQL query successfully executed. However, the result set is empty.

5. Transfer ₹ 10 from ANIL's Account to SUNIL's Account (If Both Have Same Branch)

Syntax: -- Step A: Deduct 10 Rs from Anil if he shares a branch with Sunil
UPDATE deposit
SET amount = amount - 10
WHERE cname = 'anil'
AND bname IN (SELECT bname FROM deposit WHERE cname = 'sunil');
-- Step B: Add 10 Rs to Sunil if he shares a branch with Anil
UPDATE deposit
SET amount = amount + 10
WHERE cname = 'sunil'
AND bname IN (SELECT bname FROM deposit WHERE cname = 'anil');

Output

SQL query successfully executed. However, the result set is empty.

6. Give ₹ 100 More to Depositors Having Maximum Deposit in Their Branch

```
Syntax: UPDATE deposit d
SET d.amount = d.amount + 100
WHERE d.amount = (
    SELECT MAX(amount)
    FROM deposit
    WHERE bname = d.bname);
```

Output

SQL query successfully executed. However, the result set is empty.

7. Delete Depositors of Branches Having Number of Customers Between 1 and 3

```
Syntax: DELETE FROM deposit
WHERE bname IN (
    SELECT bname
    FROM deposit
    GROUP BY bname
    HAVING COUNT(DISTINCT cname) BETWEEN 1 AND 3
);
```

Output

SQL query successfully executed. However, the result set is empty.

8.Delete Deposit of VIJAY

```
Syntax: DELETE FROM deposit
WHERE cname = 'vijay';
```

Output

SQL query successfully executed. However, the result set is empty.

9.Delete Borrowers of Branches Having Average Loan Less Than 1000

```
Syntax: DELETE FROM borrow
WHERE bname IN (
    SELECT bname
    FROM borrow
    GROUP BY bname
    HAVING AVG (amount) < 1000
);
```

Output

SQL query successfully executed. However, the result set is empty.

Practical 9

AIM: TCL and DCL Commands in SQL

-- 1. Explicitly drop the table (ignore if it throws an error that it doesn't exist)

```
DROP TABLE EMPLOYEE;
```

-- 2. Create the table fresh

```
CREATE TABLE EMPLOYEE (  
    EMP_NO INT PRIMARY KEY,  
    EMP_NAME VARCHAR(50),  
    EMP_SAL DECIMAL(10, 2)  
);
```

-- 3. Insert mock records

```
INSERT INTO EMPLOYEE (EMP_NO, EMP_NAME, EMP_SAL) VALUES  
(101, 'Anil', 25000.00),  
(102, 'Sunil', 30000.00),  
(103, 'Vijay', 28000.00),  
(104, 'Rahul', 35000.00);
```

Query:

1. COMMIT

Syntax: UPDATE EMPLOYEE

```
    SET EMP_SAL = EMP_SAL + 500
```

```
    WHERE EMP_NO = 101;
```

```
SELECT * FROM EMPLOYEE WHERE EMP_NO = 101;
```

Output

EMP_NO	EMP_NAME	EMP_SAL
101	Anil	25500

2. ROLLBACK

Syntax: BEGIN TRANSACTION;

```
UPDATE EMPLOYEE
```

```
    SET EMP_SAL = EMP_SAL + 1000
```

```
    WHERE EMP_NO = 102;
```

ROLLBACK;

SELECT * FROM EMPLOYEE WHERE EMP_NO = 102;

Output		
EMP_NO	EMP_NAME	EMP_SAL
102	Sunil	30000

3. SAVEPOINT

Syntax: BEGIN TRANSACTION;

UPDATE EMPLOYEE

SET EMP_SAL = EMP_SAL + 500

WHERE EMP_NO = 103;

SAVEPOINT S1;

UPDATE EMPLOYEE

SET EMP_SAL = EMP_SAL + 1000

WHERE EMP_NO = 104;

ROLLBACK TO S1;

COMMIT;

SELECT * FROM EMPLOYEE

Output		
EMP_NO	EMP_NAME	EMP_SAL
101	Anil	25000
102	Sunil	30000
103	Vijay	28500
104	Rahul	35000

4. GRANT

Syntax: GRANT SELECT, INSERT ON Employees TO user1;

5. REVOKE

Syntax: REVOKE INSERT ON Employees FROM user1;

Practical 10

AIM: PL/SQL Block

```

DROP TABLE IF EXISTS MATH_LAB_DATA;

CREATE TABLE MATH_LAB_DATA (
    RECORD_ID INT PRIMARY KEY,
    NUM_A NUMBER,
    NUM_B NUMBER,
    NUM_C NUMBER,
    SHAPE_TYPE VARCHAR(30)
);

INSERT INTO MATH_LAB_DATA (RECORD_ID, NUM_A, NUM_B, NUM_C, SHAPE_TYPE)
VALUES
(1, 10, 20, NULL, 'Addition'),
(2, 10, 5, NULL, 'Rectangle'),
(3, 10, 5, NULL, 'Triangle'),
(4, 8, NULL, NULL, 'Square'),
(5, 10, 20, 15, 'Max Check'),
(6, NULL, NULL, 10, 'Loop Limit');
```

Query:

1. Write a PL/SQL Block to Add 2 Numbers

Syntax: SELECT 10 AS A, 20 AS B, (10 + 20) AS SUM;

Output		
A	B	SUM
10	20	30

2. Write a PL/SQL Block to find Area of Rectangle, Triangle and Square,Area of Rectangle

Syntax: SELECT 10 AS LENGTH, 5 AS BREADTH, (10 * 5) AS AREA_OF_RECTANGLE;

Output		
LENGTH	BREADTH	AREA_OF_RECTANGLE
10	5	50

3. Write a PL/SQL Block to find Maximum of 3 Numbers

```
Syntax: SELECT 10 AS A, 20 AS B, 15 AS C,  
CASE  
    WHEN 10 >= 20 AND 10 >= 15 THEN 10  
    WHEN 20 >= 10 AND 20 >= 15 THEN 20  
    ELSE 15  END AS MAXIMUM;
```

Output

A	B	C	MAXIMUM
10	20	15	20

4. Write a PL/SQL Block to Print Sum of N Numbers using FOR Loop

```
Syntax: SELECT 10 AS N, (10 * (10 + 1)) / 2 AS SUM;
```

Output

N	SUM
10	55

5. Write a PL/SQL Block to Generate Fibonacci Series of N Numbers

```
Syntax: WITH RECURSIVE fibonacci(n, current_val, next_val) AS (  
    -- Initial two rows setup  
    VALUES (1, 0, 1)  
    UNION ALL  
    -- Loop constraint rules up to N=10 numbers  
    SELECT n + 1, next_val, current_val + next_val  
    FROM fibonacci  
    WHERE n < 10  
)  
SELECT current_val AS FIBONACCI_SERIES FROM fibonacci;
```

Output

FIBONACCI_SERIES
0
1
1
2
3
5
8
13
21
34